RELATÓRIO TÉCNICO

Grafo de Transformação de Palavras — Linguagem C

1. Identificação da Equipe

Campo	Informação
Instituição	Universidade do Estado de Santa Catarina
Disciplina	Teoria dos Grafos
Integrante(s)	Rafael Horst Ruthes, Julio Cesar Crestani Peretti, Gabriel Petrentchuk
Professor(a)	Gilmário Barbosa dos Santos
Data	19/06/2026

2. Identificação da Tarefa

Implementação, em C, de um grafo não direcionado em que os vértices são palavras de 4 letras e as arestas conectam pares de palavras que diferem em exatamente uma posição de caractere (ex.: LAGO → LOGO). O grafo é carregado a partir de uma base de palavras (palavras.txt) e analisado quanto a componentes conexos, classificação estrutural e caminho mínimo entre palavras.

Requisitos funcionais:

- (a) Implementação em linguagem C.
- (b) Carga primária do grafo a partir da base de palavras.
- (c.i) Componentes conexos, tamanho de cada componente, palavra central (grau máximo) e periférica (grau mínimo).
- (c.ii) Classificação como grafo simples ou multigrafo (com contagem de laços e arestas múltiplas).
- (c.iii) Menor sequência de transformações entre duas palavras fornecidas pelo usuário, via Dijkstra.

3. Explicação Conceitual — Entrada, Processo e Saída

Entrada: o arquivo palavras.txt é lido palavra por palavra; apenas tokens com exatamente 4 caracteres são considerados vértices. Durante a carga, palavras repetidas (mesmo ignorando maiúsculas/minúsculas) são detectadas e descartadas, garantindo que cada vértice represente uma palavra logicamente única — e não um artefato de duplicação no arquivo de origem.

Processo:

- Construção das arestas: para cada par de palavras (i, j) com i < j, verifica-se se diferem em exatamente uma posição; em caso positivo, uma aresta é criada na matriz de adjacência e os graus de ambos os vértices são incrementados.
- Componentes conexos: percorridos via Busca em Largura (BFS) com fila implementada em array; para cada componente, registra-se o tamanho e os vértices de grau máximo (palavra central) e mínimo (palavra periférica).

- Classificação do grafo: verificação estrutural da diagonal da matriz (laços) e de valores de adjacência maiores que 1 (arestas múltiplas). Como uma palavra nunca pode diferir de si mesma em exatamente uma posição, e a base já foi deduplicada na carga, o resultado correto para este domínio é sempre grafo simples — uma propriedade matemática do problema, não um valor padrão do código.
- Caminho mínimo: algoritmo de Dijkstra, com peso uniforme 1 por aresta (cada transformação de uma letra conta como um passo); reconstrução do caminho via vetor de predecessores.

Saída: menu interativo no terminal com três operações — análise de componentes conexos, classificação do grafo e busca do menor caminho entre duas palavras digitadas pelo usuário.

4. Compilação e Execução

Implementação no VS Code, utilizando apenas a biblioteca padrão C (stdio.h, stdlib.h, string.h, ctype.h, limits.h) e GCC via linha de comando. Código foi desenvolvido e testado em ambientes Windows e Ubuntu.

Compilação:

```
gcc grafo_palavras.c
```

Execução:

```
./a.out
```

O programa busca automaticamente o arquivo palavras.txt no diretório; o menu interativo permite escolher entre análise de componentes (1), classificação do grafo (2), busca de caminho mínimo (3) e saída (4).

5. Análise de Resultados

Métrica	Valor
Arquivo de entrada	palavras.txt
Vértices únicos (após deduplicar)	depende da base — ex.: 10 em base de teste
Duplicatas descartadas	reportado na carga (ex.: 2 em base de teste)
Tipo do grafo	Simples (0 laços, 0 arestas múltiplas)
Algoritmo de caminho mínimo	Dijkstra (peso uniforme = 1 por transformação)

Em uma base de teste contendo a cadeia de transformação do enunciado (LAGO, LOGO, LEGO, CEGO, CEGA, CENA) somada a palavras adicionais e duplicatas propositais de "MATA", o sistema identificou corretamente 3 componentes conexos: a cadeia principal de 6 palavras (com LEGO como palavra central, grau 3, por se conectar a LOGO, CEGO e, indiretamente na vizinhança, outras transformações), e dois componentes isolados de 2 palavras cada. As duas entradas duplicadas de "MATA" foram detectadas e descartadas na carga, evitando a inflação artificial do número de vértices.

A classificação do grafo retornou "Grafo Simples", com a justificativa estrutural de que, neste domínio, laços são matematicamente impossíveis (uma palavra não pode diferir de si mesma em exatamente uma posição) e arestas múltiplas foram eliminadas pela deduplicação prévia da base.

Para a busca de caminho mínimo entre LAGO e CENA, o algoritmo de Dijkstra encontrou a sequência LAGO → LEGO → CEGO → CEGA → CENA, totalizando 4 passos que difere do exemplo do enunciado com uma transformação intermediária a menos, uma vez que LAGO já se conecta diretamente a LEGO por diferir em apenas uma letra.

Fim do Relatório